

# Practice Exam 1

Wednesday, March 4, 2026 5:03 PM

For examples of multiple-choice questions on history and vocabulary, see the online quizzes.

## Scheme

1. Evaluate each of the following expressions.
  - a. (car (cdr '(red yellow blue green)))
  - b. (caddr '(lime kelly forest mint))
  - c. (cons (car '(purple violet)) (cdr '(orange tangerine)))
2. Draw the internal node structure for the list **(a (b c) (d) e)**
3. Assuming that function **f** has been defined as follows:

```
(define (f L)
  (cond ((null? L) 'troll)
        ((null? (cdr L)) 'elf)
        ((> (car L) (* (cadr L) 2)) 'hobbit)
        (else (f (cdr L)))))
```

evaluate the following expressions:

- a. (f '(1 5 9))
  - b. (f '(4 13 2 3))
4. Write a Scheme function (**twice Item L**) which returns #T if Item occurs twice in a row anywhere in L, and returns #F otherwise. Assume that L will be a list of atoms when the function is called. For example, (twice 'a '(b a a c d)) should return #T, (twice 'b '(b b b)) should return #T, (twice 'a '(a b a c)) should return #F, and (twice 'c '(a b)) should return #F. (Do not use *display* or any other screen-output function in your program. Your function should *return* #T or #F, not merely display the value on the screen.)

## Grammars and syntax diagrams

5. A simple loop in Ada has the following form:

```
loop
  <statement>;
  <statement>;
  ...
  <statement>;
end loop;
```

That is, it begins with the word *loop*, contains one or more statements (each followed by a semicolon), and ends with the words *end loop*. There must be at least one statement inside the loop.

- a) Write a set of Extended Backus-Naur Form (EBNF) grammar rules that describe Ada simple loops. You may assume that <statement> has already been defined elsewhere.
  - b) Write a syntax diagram that describes Ada simple loops. You may assume that **statement** has already been defined elsewhere.
6. Consider this BNF grammar:

```
<Duration> → <Time> - <Time>
<Duration> → <Time>
<Time> → <Num> : <Num> [ : <Num> ] [ <DaySegment> ]
<DaySegment> → AM
<DaySegment> → PM
<Num> → <Digit> <Digit>
<Digit> → 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

Circle all the strings in the following list that can be produced from this grammar. Assume that <Duration> is the starting symbol in the grammar.

9:45AM-10:15AM  
03:30-10:45AM  
10PM-11PM  
77:30-88:00  
11:30

10:20:04AM-11:22:33

*Nancy Tinkham*  
*Computer Science Department, Rowan University*